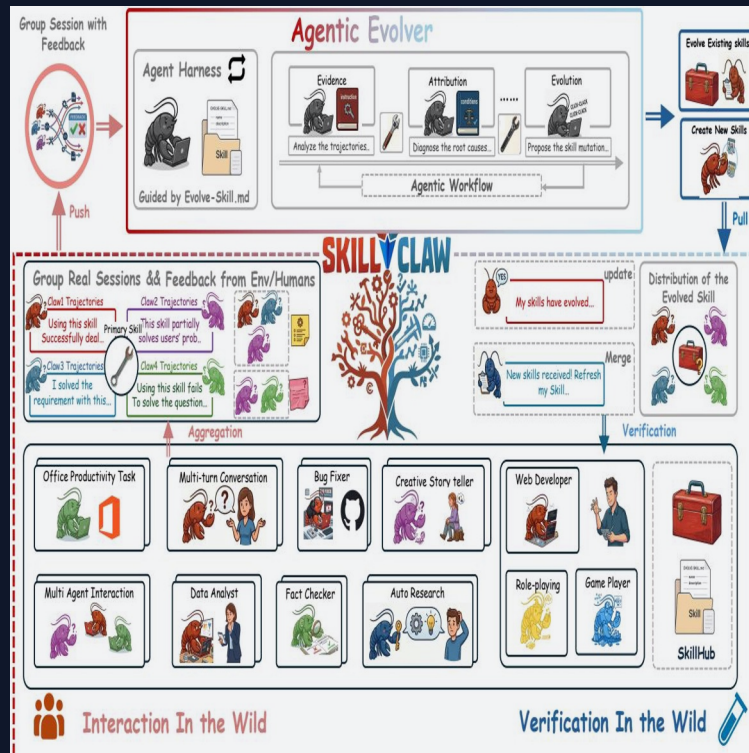


SkillClaw

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

Let Skills Evolve Collectively with Agentic Evolver

arxiv.org/abs/2604.08377



Agent Skills Remain Static After Deployment

Users Rediscover Solutions Independently

Current: Static Skills

- Skills manually installed from a hub
- Never updated from runtime experience
- Failures rediscovered across users
- No system-level knowledge accumulation
- Memory methods tied to specific instances

Needed: Evolving Skills

- Skills improve from real-world usage
- Cross-user evidence drives updates
- Solutions persist beyond sessions
- Shared knowledge accumulates
- Automatic, no manual curation

The gap: no existing method bridges static skill libraries and runtime-evolving agent capabilities

Closed-Loop Pipeline: From Multi-User Interaction to Shared Skill Evolution

Multi-User Interaction

OpenClaw · CoPaw · IronClaw · PicoClaw · ZeroClaw · NanoClaw · NemoClaw

Agentic Evolver

1

EVIDENCE

Analyze trajectories to identify behavioral patterns across sessions



2

ATTRIBUTION

Diagnose root causes of failures and successes in skill execution



3

EVOLUTION

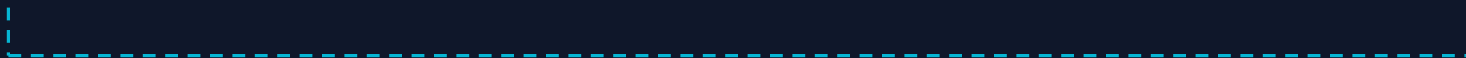
Propose skill mutations — refine existing skills or create new ones



4

DISTRIBUTION

Verify updates, merge into shared repository, sync to all agents



Skill Synchronization — Closed Loop

Agentic Evolver: Evidence → Attribution → Evolution

Three stages of open-ended agent reasoning — not predefined rules

Stage 1: EVIDENCE

Analyze Trajectories

- Collect structured session trajectories from multi-user interactions
- Preserve full action–feedback causal chains
- Group trajectories by referenced skills
- Identify behavioral patterns across sessions



Stage 2: ATTRIBUTION

Diagnose Root Causes

- Determine why a skill succeeded or failed in a given context
- Distinguish generalizable patterns from idiosyncratic fixes
- Cross-reference failure modes across different users
- Isolate actionable improvement signals



Stage 3: EVOLUTION

Propose Skill Mutations

- Refine existing skills based on attributed causes
- Create new skills when gaps are identified
- Generate candidate skills for verification
- Conservative: only verified mutations are merged

Cross-User Trajectory Aggregation Exposes Skill Behavioral Boundaries



Why Aggregation Matters

Complementary signals

Different users exercising the same skill under diverse contexts produce complementary views — revealing both success conditions and failure modes.

Single-user insufficiency

A single user rarely generates enough signal to separate a generalizable improvement from an idiosyncratic fix.

Behavioral boundary mapping

Cross-user aggregation maps the boundary where a skill works vs. where it breaks, enabling targeted evolution.

Case Study: Multimodal Creative Pipeline Skill

6 Days of Evolution Attempts — 1 of 6 Accepted

1 / 6
accepted

Day	Candidate Skill	Skill Function	Result	Next-Day Action
1	validate-tmp-workspace-inputs	Check /tmp_workspace inputs & environment setup	Accept	Upgrade to new best pool
2	multimodal-input-validation-and-setup	Multimodal input validation & output env init	Reject	Keep current best pool
3	multimodal-creative-task-pipeline	Multimodal creative pipeline (extract from PDF/video/image)	Reject	Keep current best pool
4	multimodal-creative-task-pipeline (impr.)	Added image classification, visual generation, structured validation	Reject	Keep current best pool
5	...(impr.) + validate-required-input-files	Creative pipeline + per-file fail-fast validation	Reject	Keep current best pool
6	multimodal-creative-task-pipeline (cand.)	Extended PDF-to-poster / document-to-visual paths	Reject	Not admitted to next cycle

Key insight: Conservative verification prevents regressions — only the Day 1 candidate passed validation across the full 6-day cycle.

Case Study: Git Push Auth-Fallback Skill

Iterative Refinement Over 6 Days — 3 of 6 Accepted

3 / 6
accepted

Day	Candidate Skill	Change Summary	Result	Next-Day Action
1	git-push-with-auth-fallback	Safe fallback instead of blocking on push failure	Accept	Add to Safety best pool
2	git-push-with-auth-fallback	Unified patch/bundle filenames, reduced inconsistency	Accept	Keep updated best pool
3	... + git-clone-to-directory	Auth-alternative paths & secrets audit; fixed subshell pitfalls	Accept	Keep current best pool
4	(none)	Same-pool retest; validator confirmed current pool as best	Accept	Continue same best pool
5	git-push-with-auth-fallback	Added "push hang treated as auth failure"; no improvement	Reject	Keep current best pool
6	git-push-with-auth-fallback	Added identity config & filename consistency; no improvement	Reject	Not admitted to next cycle

Key insight: Iterative refinement works when evidence supports it — the skill improved 3 times before reaching a plateau at Day 4.

Key Takeaways: Three Properties That Distinguish SkillClaw

1 Collective Evolution

Knowledge from individual interactions contributes to a shared, continuously improving skill ecosystem. Each user's experience benefits all other users.

2 Fully Automatic

Skill evolution is driven by runtime interaction without manual curation or explicit user intervention. No human in the loop required.

3 Agentic Evolution Paradigm

Skill updates are produced through open-ended reasoning over evidence rather than predefined update rules. The evolver adapts its strategy to the data.

Implications for Continuously Improving Agent Systems

SkillClaw demonstrates that agent ecosystems can self-improve by aggregating distributed interaction evidence and applying agentic reasoning to evolve shared skill repositories.

Evaluated on WildClawBench with Qwen3-Max backbone, showing substantial improvements across tasks. Compatible with OpenClaw, CoPaw, IronClaw, PicoClaw, ZeroClaw, NanoClaw, and NemoClaw.